

CAREERBRIDGE – DAY 7 EXERCISE

Registration, Login, Password Hashing and JWT Authentication

Project: CareerBridge – Job & Internship Portal

Week: 2

Day: 7

Student Name: Larbi Peniels Okyerewa

Date: 9/15/2026

PART A — AUTHENTICATION VS AUTHORIZATION

Explain the following in your own words.

Authentication

It is the process of verifying who a user is.

Authorization

It is the process of determining what an authenticated user is allowed to do.

What is the difference?

Authentication is used to identify a user while authorization is used to determine what the user is allowed to do.

Questions

What is the Primary Key?

UserID

Which field should normally be unique?

Email

Should the password itself be stored?

No

What should be stored instead?

A password hash

Complete the table:

Package	Purpose
bcryptjs	Used to hash passwords and compare passwords with their stored hashes.
jsonwebtoken	Used to create and verify JSON Web Tokens.

Question

Why should JWT_SECRET never be committed to GitHub?

It is a private secret used to sign and verify JWTs. If it is exposed, someone could potentially create or verify forged tokens.

PART G — VALIDATE THE REQUEST

Registration should not continue if required fields are missing.

Test what happens when the request contains:

```
{  
  "email": "ama@example.com"  
}
```

The API should return an appropriate client error rather than attempting to create an incomplete user.

Question

Which HTTP status code would you use?

404 – Bad Request

Explain:

The request is invalid because required information is missing. Therefore, the server rejects the request instead of creating an incomplete user.

PART H — CHECK FOR DUPLICATE EMAIL

Before inserting the user, check whether the email already exists.

Conceptually:

```
SELECT UserID
```

```
FROM tblUser
```

```
WHERE Email = @Email
```

Use a parameterized SQL query.

Do **not** construct SQL like:

```
"SELECT * FROM tblUser WHERE Email = '" + email + "'"
```

Question

Why is building SQL from raw user input dangerous?

Because it can allow SQL injection.

PART I — PASSWORD HASHING

Never store:

```
Password123!
```

directly in the database.

Instead, generate a hash using bcrypt.

Concept:

```
const bcrypt = require("bcryptjs");
```

```
const passwordHash = await bcrypt.hash(password, 10);
```

The database should contain something resembling:

\$2b\$10\$.....

rather than the original password.

Questions

Why do we hash passwords?

To prevent people from accessing them.

Can we normally convert a bcrypt hash back into the original password?

No

Why is that important?

Because password hashing is designed to be one-way. The system verifies a password by comparing it with the stored hash rather than storing the original password.

PART K — REGISTRATION RESPONSE

A successful registration could return:

```
{  
  "success": true,  
  "message": "Registration successful"  
}
```

Use an appropriate HTTP status code.

Which status code represents successful creation?

HTTP 201 - Created

PART P — COMPARE PASSWORDS

Use bcrypt:

```
const passwordMatches = await bcrypt.compare(  
  password,
```

```
    user.PasswordHash  
  );
```

Question

Why don't we hash the login password ourselves and compare the two hash strings?

Because bcrypt uses a **salt**, so hashing the same password again can produce a different hash.

PART U — TOKEN VERIFICATION

The middleware should conceptually perform:

```
const decoded = jwt.verify(  
  token,  
  process.env.JWT_SECRET  
);
```

If valid:

```
req.user = decoded;
```

Then:

```
next();
```

Question

What is the purpose of next() in Express middleware?

next() tells Express that the current middleware has finished its work and that the request should **continue to the next middleware or route**.

PART X — INTRODUCTION TO ROLES

CareerBridge has three roles:

JobSeeker

Employer

Admin

Suppose the JWT contains:

```
{  
  "userId": 5,  
  "role": "JobSeeker"  
}
```

Answer:

Should this user be allowed to post a job

No

Should an Employer be allowed to post a job?

Yes

Should a Job Seeker access the Admin dashboard?

No

What concept controls these permissions?

Authorization / role-based access control (RBAC).

PART Y — SECURITY THINKING

Consider this registration request:

```
{  
  "firstName": "Ama",  
  "lastName": "Mensah",  
  "email": "ama@example.com",
```

```
"password": "Password123!",  
"role": "Admin"  
}
```

Question

Should a normal public registration endpoint allow someone to register themselves as an Administrator?

No

Explain why:

Because users should not be able to give themselves administrator privileges. An attacker could register as Admin and gain access to administrative functions.

How would you prevent this?

Do not trust the role supplied by the public user. Automatically assign a normal registration the JobSeeker role.

PART Z — THINKING QUESTIONS

Answer in your own words.

1. What is password hashing?

Password hashing is the process of converting a password into a protected hash before storing it in the database.

2. Why should plain passwords never be stored?

Because anyone who gains access to the database could directly see the users' passwords.

3. What does bcrypt do?

Bcrypt hashes passwords and allows the application to compare an entered password with its stored hash.

4. What is JWT?

JWT stands for **JSON Web Token**. It is a token used to represent authenticated information so the server can verify a user's identity on protected requests.

5. What information should NOT be stored inside a JWT?

Passwords, Password hash, JWT secrets.

6. What is a Bearer token?

A Bearer token is an authentication token sent with a request using the Authorization header.

7. What is middleware?

Middleware is code that runs **between the incoming request and the final route**.

8. What is the difference between HTTP 401 and HTTP 403?

401 Unauthorized means the request does not have valid authentication whiles **403 Forbidden** means the user is authenticated but **does not have permission** to perform the requested action.

9. Why should JWTs expire?

JWTs should expire to reduce the amount of time a stolen or compromised token can be used.

10. Why should SQL queries use parameters?

Parameterized queries help prevent **SQL injection** and safely separate user input from the SQL command.